Paul Belleville

Programmation Python : Séance 6

Les listes : stocker, parcourir et manipuler des données

Durée indicative : 3 heures
Thème : listes, indexation, modification, parcours et traitement de données
Exercices : Python Programming MOOC - Université d'Helsinki

---

1. Objectifs de la séance

À la fin de cette séance, vous devez être capables de :

- créer une liste et y stocker plusieurs valeurs ;
- accéder à un élément à partir de son indice ;
- modifier, ajouter et supprimer des éléments ;
- parcourir une liste avec une boucle `for` ;
- connaître le nombre d'éléments d'une liste avec `len()` ;
- vérifier si une valeur appartient à une liste avec `in` ;
- effectuer des traitements simples sur les éléments d'une liste.

Prérequis

Cette séance réutilise les notions vues précédemment : variables, types de données, chaînes de caractères, conditions, boucles `for` et `while`, compteurs et accumulateurs.

---

2. Pourquoi utiliser des listes ?

Jusqu'à présent, nous avons principalement manipulé des valeurs individuelles. Imaginons que nous souhaitions stocker les prénoms de trois personnes. Nous pourrions créer trois variables :

```
name1 = "Alice"
name2 = "Bob"
name3 = "Charlie"
```

Cette solution fonctionne, mais elle devient rapidement peu pratique si nous devons stocker des dizaines ou des centaines de noms.

Une **liste** permet de regrouper plusieurs valeurs dans une seule variable :

```
names = ["Alice", "Bob", "Charlie"]

print(names)
```

Résultat :

```
['Alice', 'Bob', 'Charlie']
```

Une liste est une collection **ordonnée et modifiable** d'éléments. Elle peut contenir plusieurs valeurs, y compris des valeurs de types différents :

```
information = ["Alice", 22, True]
```

En pratique, il est généralement préférable de regrouper dans une même liste des éléments qui ont un rôle similaire.

Créer une liste vide

Il est également possible de créer une liste sans élément :

```
names = []
```

Nous pourrons ensuite la remplir au fur et à mesure de l'exécution du programme.

---

3. Accéder aux éléments d'une liste

3.1. Les indices

Chaque élément d'une liste possède une position appelée **indice** (*index*). En Python, le premier indice est `0`, et non `1`.

```
names = ["Alice", "Bob", "Charlie"]

print(names[0])  # Alice
print(names[1])  # Bob
print(names[2])  # Charlie
```

Élément	Alice	Bob	Charlie
Indice	0	1	2

3.2. Accéder au dernier élément

L'indice `-1` permet d'accéder au dernier élément d'une liste. Les indices négatifs comptent à partir de la fin.

```
names = ["Alice", "Bob", "Charlie"]

print(names[-1])  # Charlie
print(names[-2])  # Bob
```

3.3. Les erreurs d'indexation

Un indice doit correspondre à un élément existant.

```
names = ["Alice", "Bob", "Charlie"]

print(names[3])
```

Ce programme provoque une erreur : la liste contient trois éléments, dont les indices vont de `0` à `2`.

À retenir : si une liste contient `n` éléments, ses indices positifs vont de `0` à `n - 1`.

---

4. Modifier une liste

Les listes Python sont **modifiables** : nous pouvons remplacer, ajouter ou supprimer leurs éléments.

4.1. Modifier un élément

Pour remplacer un élément, il suffit d'utiliser son indice et de lui affecter une nouvelle valeur.

```
names = ["Alice", "Bob", "Charlie"]

names[1] = "David"

print(names)
```

Résultat :

```
['Alice', 'David', 'Charlie']
```

4.2. Ajouter un élément avec `append()`

La méthode `append()` ajoute un élément à la fin de la liste.

```
names = ["Alice", "Bob"]

names.append("Charlie")
```

```
print(names)
```

Résultat :

```
['Alice', 'Bob', 'Charlie']
```

On peut également ajouter plusieurs éléments successivement :

```
names = []

names.append("Alice")
names.append("Bob")
names.append("Charlie")

print(names)
```

4.3. Insérer un élément avec `insert()`

La méthode `insert()` ajoute un élément à une position précise.

```
names = ["Alice", "Charlie"]

names.insert(1, "Bob")

print(names)
```

Résultat :

```
['Alice', 'Bob', 'Charlie']
```

Le premier argument correspond à l'indice, le second à la valeur à insérer.

4.4. Supprimer un élément avec `remove()`

La méthode `remove()` supprime la première occurrence d'une valeur donnée.

```
names = ["Alice", "Bob", "Charlie"]

names.remove("Bob")

print(names)
```

Résultat :

```
['Alice', 'Charlie']
```

Attention : si la valeur n'est pas présente dans la liste, Python déclenche une erreur (`ValueError`).

4.5. Supprimer un élément avec `pop()`

La méthode `pop()` supprime un élément à partir de son indice et renvoie sa valeur.

```
names = ["Alice", "Bob", "Charlie"]

removed_name = names.pop(1)

print(removed_name)  # Bob
print(names)         # ['Alice', 'Charlie']
```

Si aucun indice n'est fourni, `pop()` supprime et renvoie le dernier élément.

```
last_name = names.pop()
```

---

5. Connaître la taille d'une liste : `len()`

La fonction `len()` renvoie le nombre d'éléments contenus dans une liste.

```
names = ["Alice", "Bob", "Charlie"]

print(len(names))
```

Résultat :

```
3
```

Elle fonctionne également avec les chaînes de caractères :

```
word = "Python"

print(len(word))  # 6
```

Parcourir les indices d'une liste

La fonction `range()` peut être combinée avec `len()` pour parcourir les indices d'une liste.

```
names = ["Alice", "Bob", "Charlie"]

for i in range(len(names)):
    print(names[i])
```

Cette solution fonctionne, mais lorsque nous avons simplement besoin des valeurs, il est souvent plus lisible d'écrire directement :

```
for name in names:
    print(name)
```

Nous allons voir cette deuxième manière plus en détail.

---

6. Parcourir une liste avec `for`

Une boucle `for` permet de parcourir un objet itérable et d'exécuter un bloc d'instructions pour chacun de ses éléments. Une liste est un exemple d'objet itérable.

```
names = ["Alice", "Bob", "Charlie"]

for name in names:
    print(name)
```

Résultat :

```
Alice
Bob
Charlie
```

À chaque répétition, la variable `name` reçoit l'élément suivant de la liste.

6.1. Parcourir une liste de nombres

```
numbers = [10, 20, 30, 40]

for number in numbers:
    print(number)
```

6.2. Combiner une boucle et une condition

Nous pouvons utiliser une condition à l'intérieur de la boucle pour ne traiter que certains éléments.

Exemple : afficher uniquement les nombres supérieurs ou égaux à 10.

```
numbers = [4, 12, 7, 19, 2, 8]

for number in numbers:
    if number >= 10:
        print(number)
```

Résultat :

```
12
19
```

6.3. Calculer une somme

Nous pouvons réutiliser le principe de l'accumulateur vu précédemment.

```
numbers = [10, 20, 30, 40]

total = 0

for number in numbers:
    total = total + number

print(total)
```

Résultat : `100`.

À chaque répétition, la valeur courante est ajoutée à `total`. On peut également utiliser l'écriture raccourcie `total += number`, qui signifie la même chose que `total = total + number`.

---

7. Vérifier la présence d'un élément : `in`

L'opérateur `in` permet de vérifier si une valeur est présente dans une liste. Le résultat est un booléen : `True` ou `False`.

```
names = ["Alice", "Bob", "Charlie"]

print("Bob" in names)     # True
print("David" in names)   # False
```

Il peut être utilisé directement dans une condition :

```
names = ["Alice", "Bob", "Charlie"]

name = input("Enter a name: ")

if name in names:
    print("This name is in the list.")
else:
    print("This name is not in the list.")
```

L'opérateur `not in` permet de tester l'inverse :

```
if "David" not in names:
    print("David is not in the list.")
```

---

8. Trier et inverser une liste

8.1. Trier avec `sort()`

La méthode `sort()` trie la liste directement.

```
numbers = [8, 3, 10, 1, 5]

numbers.sort()

print(numbers)
```

Résultat :

```
[1, 3, 5, 8, 10]
```

Pour trier dans l'ordre décroissant :

```
numbers.sort(reverse=True)
```

8.2. Inverser avec `reverse()`

La méthode `reverse()` inverse l'ordre actuel des éléments.

```
names = ["Alice", "Bob", "Charlie"]

names.reverse()

print(names)
```

Résultat :

```
['Charlie', 'Bob', 'Alice']
```

Attention : `reverse()` ne trie pas la liste. Elle inverse simplement son ordre actuel.

---

9. Quelques opérations utiles : Récapitulatif

Opération	Fonction
`len(names)`	Renvoie le nombre d'éléments
`names[0]`	Accède au premier élément
`names[-1]`	Accède au dernier élément
`names[0] = "David"`	Modifie un élément
`names.append("David")`	Ajoute un élément à la fin
`names.insert(1, "David")`	Insère un élément à un indice donné
`names.remove("Bob")`	Supprime la première occurrence de `"Bob"`
`names.pop(0)`	Supprime et renvoie l'élément à l'indice
`"Bob" in names`	Vérifie si `"Bob"` est présent
`names.sort()`	Trie la liste
`names.reverse()`	Inverse l'ordre des éléments

Il n'est pas nécessaire de mémoriser toutes ces méthodes immédiatement. L'important est de comprendre ce que permet une liste et de savoir retrouver la bonne opération lorsque vous en avez besoin.

---

10. Exercices : MOOC de l'Université d'Helsinki

Les exercices sont réalisés directement sur la plateforme du MOOC. Les tests automatiques vérifient votre solution et fournissent un retour immédiat.

Consignes :

1. Lisez attentivement l'énoncé.
2. Identifiez les données à manipuler et le résultat attendu.

3. Essayez de résoudre le problème avant de consulter une solution.
4. Exécutez les tests et lisez les messages de retour.
5. Si vous êtes bloqués, relisez les exemples du cours et testez votre programme sur un cas simple.

Série A : Premiers exercices sur les listes

Lien : Python Programming MOOC - Part 4

Avant de procéder aux exercices sur les listes, lisez attentivement et procédez à l'installation de l'environnement VScode comme décrit ici. Cette étape est nécessaire pour la validation de votre soumissions sur le site

Accessoirement vous serez également équipés de VScode, un indispensable pour le développement sur tous les langages, pas uniquement Python.

Exercices à rechercher dans la partie consacrée aux listes :

1. **The length of a list** : déterminer le nombre d'éléments d'une liste.
2. **Add items to a list** : ajouter des éléments à une liste.
3. **Addition and removal** : ajouter et supprimer des éléments.
4. **Printing a list** : afficher les éléments d'une liste.
5. **The average of a list** : calculer la moyenne des valeurs.

Série B : Parcourir et traiter des listes

Poursuivez dans la même partie du MOOC avec les exercices portant sur le parcours des listes et les traitements de leurs éléments.

Objectifs :

1. Parcourir une liste et afficher chacun de ses éléments.
2. Rechercher une valeur dans une liste.
3. Compter les éléments répondant à une condition.
4. Calculer une somme ou une moyenne.
5. Trouver une valeur maximale ou minimale.

Priorité : terminez les exercices les plus accessibles de la série A avant de passer à la série B. Il n'est pas nécessaire de terminer l'ensemble de la partie pendant la séance.

> Pour les bienfaits de votre apprentissage, vous pouvez également faire tous les exercices de la page, ainsi que la partie *More function*.

---

11. Mini-projet : Première version d'un analyseur de texte

Nous allons commencer à construire un programme qui reçoit un texte et en extrait des informations simples.

L'objectif final, sur plusieurs séances, sera de produire un petit analyseur capable de compter les mots, calculer des fréquences et identifier certaines caractéristiques du texte.

Pour cette séance, nous allons nous concentrer sur les listes et les boucles.

Essayez de réaliser chaque étape et utilisez la solution pour vous corriger, ou, si vous êtes bloqués, cherchez à comprendre le problème et la solution avant de passer à l'étape suivante.

Étape 1 : Récupérer un texte

▶ Afficher la solution

```
text = input('Enter a text: ')
print(text)
```

Étape 2 : Transformer le texte en liste de mots

La méthode `split()` permet de découper une chaîne de caractères en plusieurs éléments. Par défaut, elle sépare le texte au niveau des espaces.

▶ Solution

```
text = input("Enter a text: ")

words = text.split()

print(words)
```

Entrée :

```
Enter a text: Python is a programming language
```

Sortie :

```
['Python', 'is', 'a', 'programming', 'language']
```

Étape 3 : Compter le nombre de mots

▶ Solution

```
text = input("Enter a text: ")

words = text.split()

print("Number of words:", len(words))
```

Étape 4 : Afficher chaque mot séparément

▶ Solution

```
text = input("Enter a text: ")

words = text.split()

for word in words:
    print(word)
```

Étape 5 : Afficher uniquement les mots de plus de cinq caractères

▶ Solution

```
text = input("Enter a text: ")

words = text.split()

for word in words:
    if len(word) > 5:
        print(word)
```

Exercices d'application

À vous de compléter le programme :

1. Afficher le nombre de caractères du texte original.
2. Afficher le premier mot de la liste.
3. Afficher le dernier mot de la liste.
4. Afficher uniquement les mots qui commencent par la lettre `a`.
5. Compter le nombre de mots qui contiennent plus de cinq caractères.
6. Trouver et afficher le mot le plus long.

Indice pour le dernier exercice : vous pouvez créer une variable `longest_word`, initialisée avec le premier mot de la liste, puis parcourir les autres mots et remplacer cette variable lorsqu'un mot plus long est trouvé.

Attention : si le texte est vide, la liste `words` est vide. Les accès à `words[0]` ou `words[-1]` provoqueront alors une erreur. Pour cette première version, vous pouvez supposer que l'utilisateur saisit au moins un mot, ou ajouter une condition pour gérer ce cas.

---

12. Bilan : Ce qu'il faut retenir

- Une liste permet de stocker plusieurs éléments dans une seule variable.
- Les listes sont ordonnées et modifiables.
- Le premier élément possède l'indice `0`.

- Les indices négatifs permettent d'accéder aux éléments depuis la fin.
- `append()` ajoute un élément à la fin de la liste.
- `remove()` supprime une valeur ; `pop()` supprime un élément à partir de son indice et renvoie sa valeur.
- `len()` renvoie le nombre d'éléments.
- Une boucle `for` permet de parcourir les éléments d'une liste.
- `in` permet de vérifier si une valeur est présente.
- `sort()` trie une liste ; `reverse()` inverse son ordre actuel.
- `split()` permet notamment de transformer une chaîne de caractères en liste de mots.

Questions de vérification

1. Pourquoi les indices d'une liste commencent-ils à `0` en Python ?

   ▶ Afficher la réponse

   En Python, comme dans beaucoup de langages de programmation, les indices commencent à `0` car ils représentent le décalage (*offset*) par rapport au début de la liste.

   Ainsi, le premier élément se trouve à l'indice `0`, le deuxième à l'indice `1`, etc.

2. Quelle est la différence entre `append()` et `insert()` ?

   ▶ Afficher la réponse
   - `append()` ajoute un élément à la fin d'une liste.
   - `insert()` ajoute un élément à une position précise, définie par son indice.

   Exemple :

   ```
   numbers = [1, 2, 3]

   numbers.append(4)      # [1, 2, 3, 4]
   numbers.insert(1, 5)   # [1, 5, 2, 3, 4]
   ```

3. Quelle est la différence entre `remove()` et `pop()` ?

   ▶ Afficher la réponse
   - `remove()` supprime la première occurrence d'une valeur spécifiée.
   - `pop()` supprime et renvoie l'élément situé à un indice donné. Si aucun indice n'est fourni, il supprime le dernier élément.

   Exemple :

   ```
   numbers = [1, 2, 3, 2]

   numbers.remove(2)   # [1, 3, 2]
   numbers.pop(1)      # Supprime et renvoie 3
   ```

4. Que renvoie `len(["a", "b", "c"])` ?

   ▶ Afficher la réponse

   La fonction `len()` renvoie le nombre d'éléments contenus dans une liste.

   Ici, elle renvoie `3`, car la liste contient trois éléments.

5. *Sans l'exécuter*, que va afficher ce programme ?

   ```
   numbers = [2, 5, 8, 11]

   for number in numbers:
       if number > 5:
           print(number)
   ```

   ▶ Afficher la réponse

   Le programme affiche :

   ```
   8
   11
   ```

   La boucle `for` parcourt chaque élément de la liste. La condition `number > 5` vérifie si sa valeur est strictement supérieure à `5`.

   Seuls `8` et `11` satisfont cette condition.

6. Quelle est la différence entre `sort()` et `reverse()` ?

   ▶ Afficher la réponse

   - `sort()` trie les éléments d'une liste. Par défaut, le tri est effectué par ordre croissant.
   - `reverse()` inverse simplement l'ordre des éléments, sans effectuer de tri.

   Exemple :

   ```
   numbers = [3, 1, 2]

   numbers.sort()      # [1, 2, 3]

   numbers.reverse()   # [3, 2, 1]
   ```

   Attention : ces deux méthodes modifient directement la liste et ne renvoient pas une nouvelle liste.

7. Que fait `split()` lorsqu'on l'utilise sur une chaîne de caractères ?

   ▶ Afficher la réponse

La méthode `split()` découpe une chaîne de caractères en plusieurs éléments et renvoie une liste contenant ces éléments.

Par défaut, le découpage s'effectue à chaque espace (ou groupe de caractères blancs).

Exemple :

```
sentence = "Bonjour tout le monde"
words = sentence.split()

print(words)
```

Résultat :

```
['Bonjour', 'tout', 'le', 'monde']
```

On peut également préciser un séparateur :

```
date = "09/11/2026"
parts = date.split("/")
```

Résultat : `['09', '11', '2026']`.

Ressources

- Python Programming MOOC 2026 --- Université d'Helsinki
- Part 4 - Lists